3 November 2026

Struktur Data

Rekursi: Konsep & Implementasi



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta

Definisi Rekursi

Rekursi adalah **teknik pemrograman** di mana **fungsi memanggil dirinya sendiri**, baik **langsung** maupun **tidak langsung**, untuk menyelesaikan masalah besar dengan **membaginya** menjadi masalah yang **lebih kecil** dan serupa. Proses ini **berlanjut hingga** mencapai **kasus dasar (base case)** yang dapat diselesaikan langsung tanpa pemanggilan ulang. Intinya, rekursi mendefinisikan permasalahan dalam bentuk dirinya sendiri.

Contoh sederhana, **faktorial dari n (ditulis n!)** didefinisikan sebagai:

n! = n × (n-1)! jika n > 1 // Rumus rekursif memanggil nilai sebelumnya.

1! = 1 sebagai base case // Base case yang menghentikan rekursi.

Struktur Rekursi

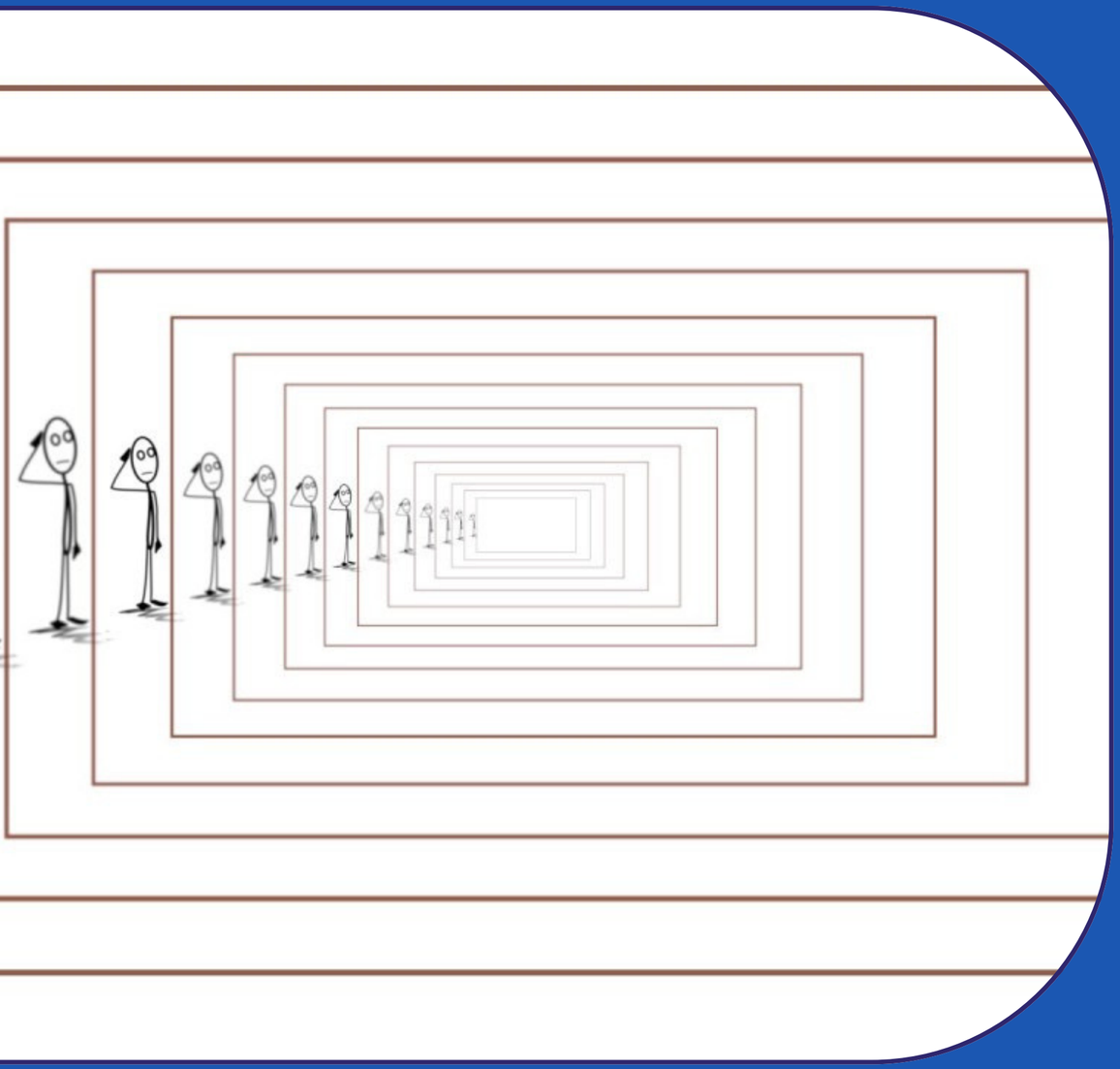
Struktur dasar dari suatu **fungsi rekursif** memiliki **dua komponen** penting:

1. **Base Case (Kasus Dasar)**: kondisi yang menghentikan rekursi. Tanpa base case, pemanggilan fungsi berlanjut terus hingga terjadi stack overflow.

2. **Recursive Case (Kasus Rekursif)**: memecah masalah menjadi lebih kecil dan memanggil fungsi sendiri.

Struktur umum fungsi rekursif:

```
tipe_data fungsi(parameter) {
    if (kondisi_base_case) {
        // solusi langsung
        return nilai;
    } else {
```



Contoh Implementasi Rekursi

1. Rekursi Faktorial
2. Rekursi Fibonacci

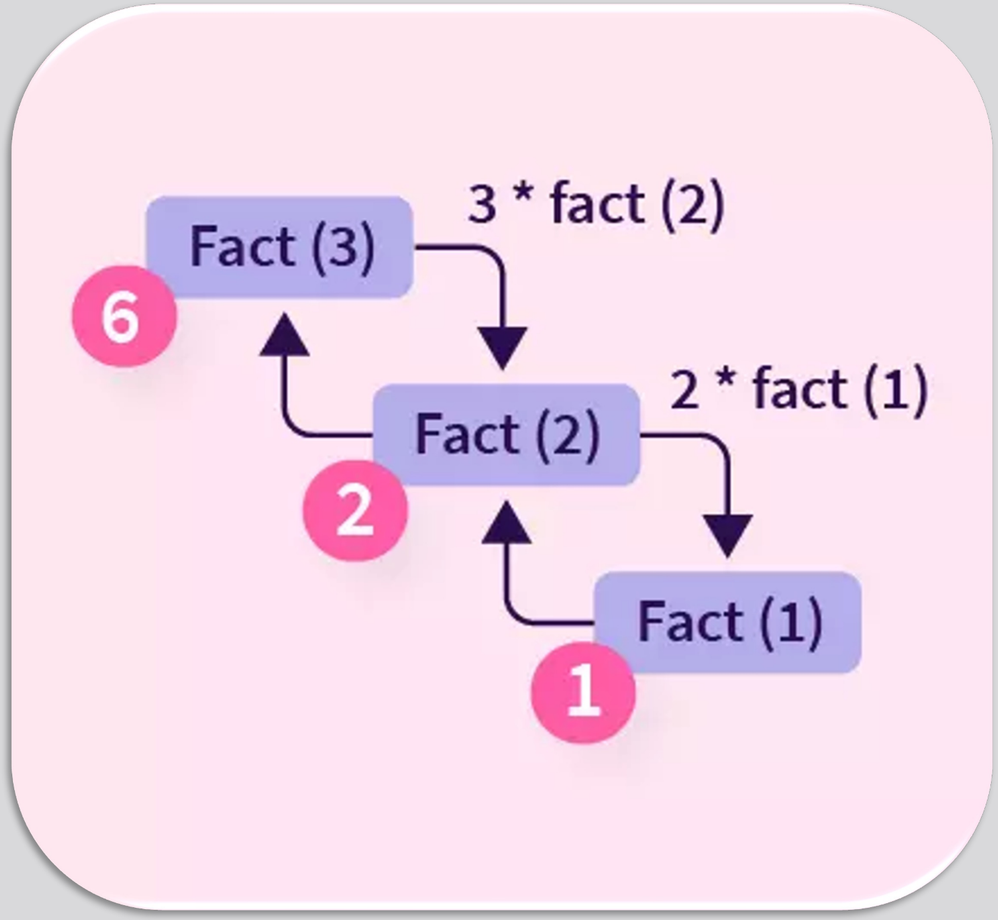
Rekursi Faktorial

Salah satu **contoh klasik** dari **rekursi** adalah **fungsi faktorial**. Faktorial dari sebuah angka n (dilambangkan sebagai n!) adalah **hasil perkalian** dari **semua bilangan** bulat positif hingga n.

Rumus Faktorial:

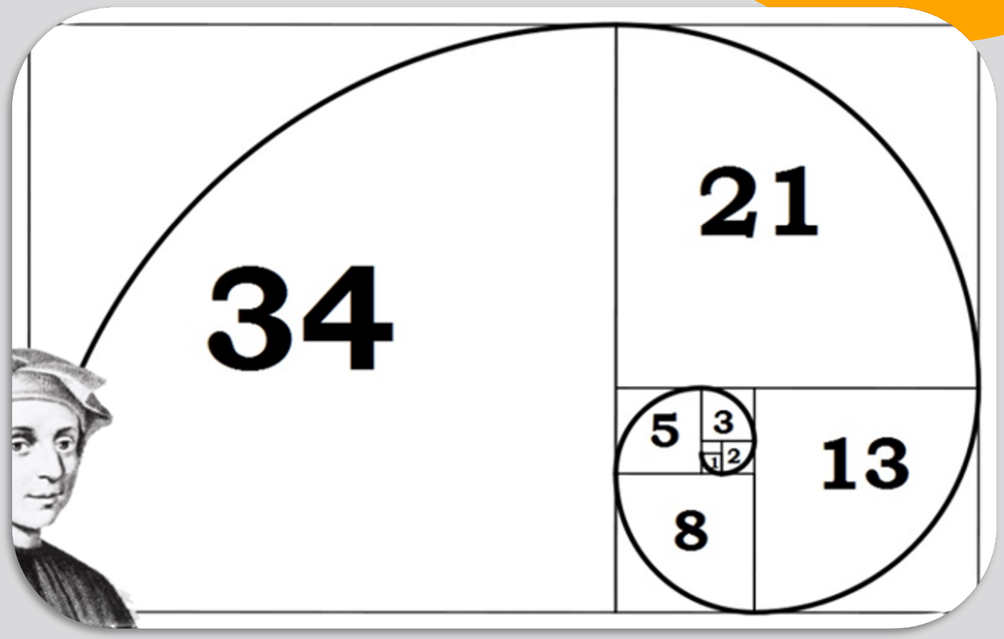
$n! = n * (n - 1)!$ untuk $n > 0$

$0! = 1$ (kasus dasar)



Rekursi Fibonacci

Deret Fibonacci adalah **urutan angka** yang dimulai dengan **0 dan 1**, di mana setiap **angka berikutnya** adalah **jumlah** dari **dua angka sebelumnya**.



Rumus Deret Fibonacci:

$F(0) = 0$

$F(1) = 1$

$F(n) = F(n-1) + F(n-2)$ untuk $n > 1$



Cara Kerja
Rekursi

1. Pemanggilan Fungsi dengan Argumen yang Lebih Kecil

2. Penyimpanan Informasi dalam Call Stack

3. Pencapaian Base Case dan Proses Backtracking

4. Kombinasi Hasil untuk Mendapatkan Solusi Akhir

Cara Kerja Rekursi

1. **Pemanggilan Fungsi dengan Argumen yang Lebih Kecil**
   - ✓ Fungsi **rekursif memecah masalah besar** menjadi **masalah** yang lebih **kecil**.
   - ✓ Setiap **pemanggilan** baru menggunakan **argumen** yang lebih **sederhana**, mendekati kondisi **base case**.
   - ✓ **Contoh**: pada perhitungan faktorial faktorial(5) akan memanggil faktorial(4), lalu faktorial(3), dan seterusnya.
   - ✓ **Ilustrasi**: faktorial(5) → 5 × faktorial(4) → 5 × (4 × faktorial(3)) → … hingga faktorial(1).

Cara Kerja Rekursi

2. **Penyimpanan Informasi dalam Call Stack**
   - ✓ Setiap **pemanggilan fungsi** menyimpan **parameter**, alamat **instruksi** berikutnya, dan **variabel lokal** sementara di **call stack**.
   - ✓ **Stack ini bekerja dengan prinsip LIFO (Last In, First Out)**: pemanggilan terakhir akan diselesaikan terlebih dahulu.
   - ✓ Dengan adanya **call stack**, program tahu harus **kembali** ke **titik pemanggilan semula** setelah **rekursi selesai**.
   - ✓ **Contoh**: Saat faktorial(5) dipanggil, call stack menyimpan setiap pemanggilan menunggu hasil dari pemanggilan sebelumnya, hingga mencapai faktorial(1) sebagai base case.

Cara Kerja Rekursi

3. Pencapaian Base Case dan Proses Backtracking

- ✓ **Rekursi berlanjut** hingga mencapai **base case**, kondisi paling sederhana yang dapat **diselesaikan langsung tanpa pemanggilan fungsi** lagi.

- ✓ Setelah **base case ditemukan**, fungsi mulai **mengembalikan nilai** ke **pemanggil** sebelumnya.

- ✓ Proses ini disebut **backtracking**, yaitu **pengembalian hasil** secara **berurutan** dari **bawah** ke **atas** tumpukan.

- ✓ **Contoh**: faktorial(1) mengembalikan 1, lalu hasil ini dipakai oleh faktorial(2) untuk menghitung $2 \times 1 = 2$, dan seterusnya hingga faktorial(5) mendapatkan hasil akhirnya.

Cara Kerja Rekursi

4. **Kombinasi Hasil untuk Mendapatkan Solusi Akhir**
   - ✓ Setiap **fungsi rekursif** yang selesai **dieksekusi** akan **mengembalikan nilai** ke **pemanggilnya**.
   - ✓ Hasil-hasil ini akan **dikombinasikan** sesuai dengan **operasi** yang didefinisikan dalam **fungsi**.
   - ✓ Pada akhirnya, **nilai** dari **pemanggilan pertama (root call)** akan menjadi **jawaban akhir** dari masalah yang diberikan.
   - ✓ **Contoh faktorial**: 1! = 1, 2! = 2, 3! = 6, 4! = 24, 5! = 120; setiap nilai dikalikan faktorial sebelumnya.

Kelebihan Rekursi

- ✓ **Kesederhanaan dan Elegan**: Rekursi seringkali menghasilkan solusi yang lebih bersih dan mudah dipahami.

- ✓ **Mengurangi Kode**: Dalam banyak kasus, fungsi rekursif memungkinkan untuk menyelesaikan masalah dalam lebih sedikit kode dibandingkan dengan menggunakan iterasi.

- ✓ **Solusi untuk Masalah Kompleks**: Rekursi sangat berguna untuk masalah yang dapat dibagi menjadi sub-masalah yang lebih kecil.

Kekurangan Rekursi

- ✓ **Penggunaan Memori**: Setiap panggilan fungsi tambahan disimpan di call stack, yang dapat menyebabkan penggunaan memori berlebih. Dalam kasus rekursi yang dalam, ini dapat menyebabkan stack overflow.

- ✓ **Kinerja Efisien**: Beberapa algoritma rekursif (seperti Fibonacci di atas) mungkin tidak efisien untuk input besar, karena mereka menghitung nilai yang sama berulang kali. Pendekatan ini dapat dioptimalkan dengan menggunakan teknik memoization atau iterasi.

- ✓ **Debugging yang Lebih Sulit**: Mem-debug kode rekursif seringkali lebih rumit dibandingkan kode iteratif.

Teknik Optimasi Rekursi

Rekursi mempermudah kode, tapi bisa **inefisien** karena **perhitungan berulang** meningkatkan eksekusi. **Memoization** adalah **teknik optimisasi rekursif** dengan **menyimpan hasil sebelumnya** di **cache**, sehingga pemanggilan ulang tidak menghitung ulang, cukup **mengambil hasil tersimpan**. **Cara Kerja Memoization**:

- ✓ **Fungsi** dipanggil dengan **argumen** tertentu.
- ✓ Jika **hasil** sudah ada di **cache**, langsung **dikembalikan**.
- ✓ Jika belum ada, **fungsi** menghitung **hasil**, menyimpannya di **cache**, lalu mengembalikan **nilai**.
- ✓ **Pemanggilan berikutnya** dengan **argumen** sama cukup mengambil dari **cache**, menghindari **perhitungan ulang**.

Teknik Optimasi Rekursi

Karakteristik Memoization:

- ✓ **Berbasis Rekursi**: Umumnya digunakan pada fungsi rekursif yang melakukan perhitungan berulang terhadap argumen yang sama.
- ✓ **Menggunakan Cache (penyimpanan sementara)**: Cache dapat berupa array, tabel hash, atau struktur data lain.
- ✓ **Mengurangi Redundansi**: Menghindari perhitungan berulang yang tidak perlu.
- ✓ **Meningkatkan Efisiensi**: Kompleksitas waktu bisa turun drastis, misalnya dari O(2^n) menjadi O(n) pada masalah Fibonacci.

Rekursi & Iterasi

- ✓ **Sederhana vs Kompleks**: Rekursi sangat cocok untuk menyelesaikan masalah yang bersifat kompleks dan hierarkis, seperti tree traversal atau algoritma divide and conquer. Kode rekursif cenderung lebih ringkas dan mudah dibaca. Iterasi lebih cocok untuk masalah sederhana, karena menggunakan loop langsung, sehingga lebih efisien dan mudah dikontrol.

- ✓ **Penggunaan Memori**: Rekursi membutuhkan ruang di call stack untuk setiap pemanggilan fungsi, berisiko stack overflow jika kedalaman terlalu besar. Iterasi lebih hemat memori karena tidak menumpuk pemanggilan fungsi, aman untuk perulangan besar.

- ✓ **Efisiensi Waktu**: Rekursi memiliki overhead pemanggilan fungsi sehingga lebih lambat. Iterasi berjalan langsung dalam loop, lebih cepat.

3 November 2026

Sekian

Terima Kasih



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta